

SinglyLinkedList.java

Objective: To create a singly linked list and add functionality to manipulate the list.

Background:

A linked list is a linear collection of data that stores data in a simple, regular structure. It consists of nodes in which each node contains a value and a link (pointer) to the next node. The last node in the list points to **null**. The advantage of linked lists is they make it easy to insert, remove, and reorder data without the data being contiguous in memory. Linked lists can grow and shrink dynamically, and can be used to implement other data structures such as queues and stacks.

Assignment:

Download files from Mr Greenstein's web site. The files include **Coordinate.java**, **ListNode.java**, **SinglyLinkedList.java**, and **SinglyLinkedListTester.java**. Create a directory **SinglyLinkedList** and do all of your work in this directory.

You will add the following functionality to the **SinglyLinkedList** class. Use the **ListNode** class for nodes. Use **SinglyLinkedListTester.java** to test your class. The data type **E** must be a **Comparable** to work properly. The **Coordinate** class implements **Comparable** and has been provided for testing. Be sure to adjust your list's **head** and **tail** pointers in each method as necessary.

1. **boolean addLast (E value)** Adds **value** to the end of the list and returns **true** if the add is successful.
2. **boolean addFirst (E value)** Adds **value** to the beginning of the list and returns **true** if the add is successful.
3. **void clear()** Clears the list of nodes.
4. **ListNode<E> getHead()** Returns the head node.
ListNode<E> getTail() Returns the tail node.
5. **int size()** Returns the number of nodes in the list.
6. **int indexOf (E obj)** Searches for the first occurrence of **obj**. If found, it returns the index of the object. If not found, it returns **-1**.
7. **E set (int index, E value)** Sets the **index** node to **value** and returns the original value in the node. If **index** is not in the list, then it throws **java.util.NoSuchElementException**.
8. **E remove (int index)** Removes the **index** node from the list and returns the **value** it removed. Throws **java.util.NoSuchElementException** if the **index** does not exist in the list.
9. **boolean add (int index, E value)** Inserts **value** into the **index** location in the list. If **index** is one greater than the largest index of the list, then the **value** is added at the end of the list. If **index** is not in the list, then it throws **java.util.NoSuchElementException**.
10. **boolean isEmpty ()** Returns **true** if the list is empty; **false** otherwise.
11. **boolean contains (E value)** Returns **true** if the list contains at least one **value**; **false** otherwise.
12. **public SinglyLinkedList(SinglyLinkedList oldList)** Implement a copy constructor for **SinglyLinkedList**. Each value in the list must be a copy of the original.